Spookython 2026

MISIÓN PUBLICADA · SPOOKYTHON 2026
PUBLICADO | Asignado 26 oct 2026, 12:01 a. m.

TRACK B · LA PLAGA ZOMBIE

Una plaga se está extendiendo por Manta. Empezó con un solo caso y cada persona mordida contagia a quienes tiene cerca: en la casa, en el bus, en el colegio, en el mercado. No hay vacunas para todos, no se puede cerrar toda la ciudad y cada día hay más zombies. Cada equipo es el Comité de Contención: construye la herramienta que le diga a la ciudad qué hacer, dónde y cuándo para que la plaga alcance a la menor cantidad de gente posible.

LA PROBLEMÁTICA REAL: Cuando llega un brote, las autoridades de salud deciden rápido y con recursos limitados: a quién vacunar primero, qué lugares cerrar, dónde fumigar. La COVID-19 lo dejó claro, y en Manabí el dengue lo recuerda cada temporada de lluvias.

UTILIDAD REAL: Es, en pequeño, lo que usan los ministerios de salud para planificar vacunación, fumigación o cierres de escuelas. El mismo motor sirve para frenar la desinformación, contener virus informáticos o hacer viral un producto.

01. OBJETIVO

Construir una herramienta que simule cómo se propaga una enfermedad en una población y encuentre la mejor estrategia de intervención con recursos limitados, mostrando con números cuánta gente se salva frente a no hacer nada o actuar al azar.

La organización NO entrega datos, vacunas ni enfermedades: cada equipo diseña su escenario y su solución. Se evalúa qué tan bien piensan el problema y qué tan sólida es su respuesta.

02. REQUISITOS

CADA EQUIPO DECIDE:

- **La plaga:** inspirada en una enfermedad real (dengue, influenza, sarampión, COVID, cólera) o inventada; transmisión, incubación, contagio, si mata o convierte.
- **La población:** red de contactos de Manta generada por ellos (hogares, colegios, buses, mercados, barrios) o con datos abiertos (INEC, Ministerio de Salud).
- **El modelo:** SIR, SEIR, basado en agentes, sobre grafo o por barrios.
- **Las intervenciones:** vacunas, cuarentenas, cierre de lugares, fumigación, antídotos, campañas.
- **Los recursos:** cuántos hay, si llegan por días, cuánto cuestan.
- **La estrategia:** centralidad en el grafo, algoritmos voraces, optimización, simulaciones, aprendizaje por refuerzo o heurística propia.

ESTADOS SUGERIDOS:

- Sano (susceptible)
- Mordido (expuesto)
- Zombie (infeccioso)
- Exorcizado (recuperado o inmune)

EL SISTEMA DEBE:

1. Simular la propagación día a día.
2. Aplicar al menos una intervención con recursos limitados.
3. Comparar su estrategia contra dos líneas base: no intervenir e intervenir al azar con los mismos recursos.
4. Repetir la simulación muchas veces y reportar promedios y rangos.
5. Tener una interfaz para cambiar los parámetros principales (contagio, vacunas, punto de inicio) y ver el resultado.
6. Ser reproducible: misma configuración y semilla, mismo resultado.
7. Estar desplegado en AWS.
8. Documentar en el README cada supuesto y de dónde salen los números.

NIVEL BASE:

- Modelo SIR o SEIR sobre una red de contactos, una intervención (por ejemplo K vacunas) y una estrategia razonable comparada con las dos líneas base.
- Gráfica de la curva de contagios y resumen de cuántos se salvan.

NIVEL PRO (puntos extra):

- Varias intervenciones combinadas con costos y un presupuesto total.
- Recursos que llegan por días y decisiones que cambian según el brote.
- Miles de simulaciones en paralelo en AWS (Monte Carlo).
- Calibración con datos reales (por ejemplo dengue en Manabí).
- Mapa animado del avance de la plaga.
- Informe para el alcalde en lenguaje simple.

IDEAS DE DIRECCIÓN (opcionales):

Vacunación estratégica, reparto de antídotos, cuarentena inteligente, plaga por mosquito con brigadas de fumigación, rastreo del paciente cero, hospital saturado.

03. REGLAS

ENTORNO:

- Cada equipo DEBE trabajar en una cuenta AWS propia (o la sandbox que entregue la organización) en la región us-east-1 (por confirmar).
- La cuenta DEBE tener un AWS Budget con alerta desde el primer día (captura en el README).
- Se permite usar asistentes de IA para programar, pero el equipo DEBE poder explicar cualquier parte de su código.
- Lenguajes y frameworks libres.

CÓMO LEER EL RETO:

- **DEBE:** obligatorio. Si no se cumple, ese criterio vale 0.
- **DEBERÍA:** esperado. No cumplirlo resta, pero no anula.
- **PUEDE:** opcional. Suma puntos si está bien hecho.

RÚBRICA BASE (sobre 10):

Criterio	Porcentaje	Descripción
Prueba de fuego	40%	Resultado de la prueba del track, en escala 0–10.
Calidad técnica	25%	Arquitectura adecuada, seguridad básica (sin llaves en el código, mínimo privilegio), código legible.
Producto	15%	Que una persona real pueda usarlo sin explicación.
Costos	10%	Declaración de costos coherente con lo que muestra la cuenta.
Presentación	10%	Claridad, respeto del tiempo y respuestas a preguntas.

En caso de empate gana el equipo con mejor prueba de fuego; si persiste, el de menor costo declarado.

PRUEBA DE FUEGO (Track B):

1. **Presentación del escenario:** plaga, población, supuestos y estrategia.
2. **Verificación:** el jurado corre la simulación en vivo con una semilla que elige y comprueba que la estrategia supera a las líneas base.
3. **Brote Sorpresa:** el jurado cambia las condiciones (por ejemplo: el doble de contagiosa, la mitad de las vacunas, el brote empieza en el mercado, una variante que la vacuna cubre a medias). El equipo tiene 5 minutos para ajustar y mostrar la nueva recomendación.

Puntuación (0–10):

- Impacto de la estrategia: 30 %
- Rigor del modelo: 25 %

- Brote Sorpresa: 25 %
- Claridad para decidir: 20 %

LO QUE EL JURADO VA A BUSCAR (errores comunes a evitar):

- Mostrar una sola simulación con buen resultado (puede ser suerte).
- No tener línea base.
- Parámetros inventados sin explicar.
- Vacunar a quien ya está infectado o es inmune.
- Un modelo tan complejo que el equipo no sabe explicar.
- Estrategias que se rompen con el Brote Sorpresa.

04. ENTREGABLES

Todo se entrega en la plataforma (Mi equipo → Entrega) hasta el miércoles 28 de octubre a las 10:00 (hora de Ecuador). Lo que llegue después no se evalúa.

LISTA DE ENTREGABLES:

- **Repositorio público** (GitHub, GitLab o Bitbucket) con el código completo y un historial de commits real, no un solo commit final.
- **README en la raíz con:** qué hace el proyecto, cómo desplegarlo, cómo probarlo, decisiones de diseño, limitaciones conocidas y la captura del AWS Budget con alerta.
- **Diagrama de arquitectura** dentro del repositorio y enlazado desde el README: servicios de AWS usados, flujo de datos y qué partes son públicas. Herramienta libre: draw.io (diagrams.net), Excalidraw, Lucidchart, Cloudcraft, AWS Infrastructure Composer o Mermaid. Suban la imagen (PNG o SVG) y, si existe, el archivo fuente (por ejemplo .drawio).
- **Declaración de costos** en el README o en `docs/costos.md`: cuánto costaría operar el sistema al mes, con el cálculo visible, y cuánto se gastó realmente durante el evento.
- **Aplicación desplegada en AWS** con una URL pública que funcione hasta el día del pitch.
- **Presentación del pitch** (PDF, PPT o PPTX, máximo 20 MB).
- **Video de demostración (opcional):** enlace público.

- **Track B:** parámetros por defecto y semilla documentados en el README para reproducir los resultados.

ESTRUCTURA SUGERIDA DEL REPOSITORIO:

README.md

docs/arquitectura.png (o .svg) y docs/arquitectura.drawio

docs/costos.md

docs/budget.png

src/

infra/ (opcional: CDK, SAM o Terraform)

DEMO EN VIVO:

Viernes 30 de octubre en la ULEAM: 5 minutos de pitch + 3 minutos de preguntas del jurado.

05. RECURSOS

SERVICIOS AWS SUGERIDOS (no hay arquitectura obligatoria):

- **Correr simulaciones:** Lambda, ECS Fargate o AWS Batch.
- **Miles de simulaciones en paralelo:** Step Functions (Distributed Map) o AWS Batch.
- **Guardar resultados:** S3 + Athena, o DynamoDB.
- **Interfaz:** Amplify o S3 + CloudFront.
- **Análisis y modelos:** SageMaker (opcional).
- **Informe en lenguaje simple:** Bedrock u otro modelo de lenguaje (opcional).
- **Monitoreo:** CloudWatch.

RECURSOS EXTERNOS:

ENLACES DE LA MISIÓN